Online Training Program
01 ASP.NET Core MVC Training
02 ASP.NET Web API Training

ASP.NET WEB API Basics
03 Introduction to ASP.NET Web API
04 Creating ASP.NET Web API Application
05 Understanding ASP.NET Web API Application
06 Understanding Routing in ASP.NET Web API
07 Creating Simple Web Service using ASP.NET Web API
08 Create Type Formatter in Web API
09 How to implement JSP Pattern in Web API
10 ASP.NET Web API using SQL Server
11 Consume Dependency in Web API
12 How to Implement TLS/SSL in Web API
13 How to implement OAUTH in Web API
14 Consume Web API using Ajax
15 Consume Method Parameter in Web API
16 Consume Web Service in Web API
17 Consuming Web API Service using Ajax
18 Calling Web API Service in a Console Application
19 Consume Web Service using Ajax
20 Create Simple Message Handler using ASP.NET Web API

ASP.NET Hosting and Deployment Tutorials
21 Hosting Web API Web Services
22 Hosting Operations in ASP.NET Web API Web Services
23 ASP.NET Web API Web Hosting Options
24 Exchange Parameters in Web API Using Post Method
25 Web API Exceptions Handling
26 Web API Authentication Using Basic Authentication
27 Client Validation Using Basic Authentication in Web API
28 Consume Web Service using Ajax
29 Create Custom Web Service Method Using Ajax
30 Consume Web Service using Ajax
31 Client Validation Using Basic Authentication in Web API
32 Consume Web Service using Ajax
33 Consume Web Service using Ajax
34 Consume Web Service using Ajax
35 Consume Web Service using Ajax
36 Consume Web Service using Ajax
37 Consume Web Service using Ajax
38 Consume Web Service using Ajax
39 Consume Web Service using Ajax
40 Consume Web Service using Ajax

Advanced Concepts
41 Handle CRUD in Visual Studio using Web API Service
42 Consume HTTPS in Web API

Projects ASP.NET Web API
43 Projects
44 Web Services Interview and ASP.NET Books

Client Validation Using Basic Authentication in Web API

Posted by: DOTNET TRICKS on Thursday, August 25, 2016

Client Validation Using Basic Authentication in Web API

In this article, we are going to discuss how to implement **Client Validation Using Basic Authentication in Web API**. Please read our previous article series prepared by Rick and Jon as we are going to use the same example. In our last article, we have discussed how to implement Form Based Authentication in ASP.NET Web API.

If you observed in the last article, we have created the following AuthorizationAttributeFilterFolder class.

```
public class AuthorizationAttributeFilterFolder : AuthorizationFilterAttribute
{
    public override void OnAuthorization(HttpActionContext actionContext)
    {
        base.OnAuthorization(actionContext);
    }
    public override bool AllowMultiple()
    {
        return true;
    }
}
```

The first method is **OnAuthorization** method is responsible for validating the Client. In the above example, we assume that have only one client and always return true in validation scenarios.

Let's change the implement. Assume that we have more than one client, also is going to consume our service. In such a case, we need to implement the Client validation Basic Authentication method.

Let's Add Authorization Filter

First thing, we are going to use the following ClientBasic class.

```
ClientBasic
ClientBasicAuthenticationFilter
ClientBasicAuthenticationFilterFolder
ClientBasicAuthenticationHandler
ClientBasicAuthenticationHandlerFolder
ClientBasicAuthenticationMessageHandler
ClientBasicAuthenticationMessageHandlerFolder
```

Please see below files. Build a new class and provide the ClientBasic name and save the files.

```
using System;
using System.Text;
namespace ClientBasicAuthentication
{
    public class ClientBasicAuthentication : IEquatable<ClientBasicAuthentication>
    {
        public ClientBasicAuthentication(Encoding encoding, string clientId, string clientSecret)
        {
            if (encoding == null)
                throw new ArgumentNullException("encoding");
            if (clientId == null)
                throw new ArgumentNullException("clientId");
            if (clientSecret == null)
                throw new ArgumentNullException("clientSecret");
            if (clientId.Contains(":"))
                throw new ArgumentException("clientId cannot contain the separator character ':'", "clientId");
            _encoding = encoding;
            _clientId = clientId;
            _clientSecret = clientSecret;
            _credentialsCache = null;
            _hashCode = 0;
            _hashCodeCalculated = false;
        }
        public ClientBasicAuthentication(ClientBasicAuthentication other)
            : this(other._encoding, other._clientId, other._clientSecret)
        {
            if (other == null)
                throw new ArgumentNullException("other");
            _credentialsCache = other._credentialsCache;
            _hashCode = other._hashCode;
            _hashCodeCalculated = other._hashCodeCalculated;
        }
        private Encoding _encoding;
        private string _clientId;
        private string _clientSecret;
        private byte[] _credentialsCache;
        private int _hashCode;
        private bool _hashCodeCalculated;
        public Encoding Encoding
        {
            get
            {
                return _encoding;
            }
            set
            {
                if (value == null)
                    throw new ArgumentNullException("value");
                _encoding = value;
                _credentialsCache = null;
                _hashCode = 0;
                _hashCodeCalculated = false;
            }
        }
        public string ClientId
        {
            get
            {
                return _clientId;
            }
            set
            {
                if (value == null)
                    throw new ArgumentNullException("value");
                if (value.Contains(":"))
                    throw new ArgumentException("clientId cannot contain the separator character ':'", "value");
                _clientId = value;
                _credentialsCache = null;
                _hashCode = 0;
                _hashCodeCalculated = false;
            }
        }
        public string ClientSecret
        {
            get
            {
                return _clientSecret;
            }
            set
            {
                if (value == null)
                    throw new ArgumentNullException("value");
                _clientSecret = value;
                _credentialsCache = null;
                _hashCode = 0;
                _hashCodeCalculated = false;
            }
        }
        public byte[] Credentials
        {
            get
            {
                if (_credentialsCache == null)
                    _credentialsCache = _encoding.GetBytes(_clientId + ":" + _clientSecret);
                return _credentialsCache;
            }
            private set
            {
                _credentialsCache = value;
                _clientId = _encoding.GetString(value).Split(':').First();
                _clientSecret = _encoding.GetString(value).Substring(_clientId.Length + 1);
                _hashCode = 0;
                _hashCodeCalculated = false;
            }
        }
        public bool Equals(ClientBasicAuthentication other)
        {
            if (other == null)
                return false;
            if (_encoding != other._encoding)
                return false;
            if (_clientId != other._clientId)
                return false;
            if (_clientSecret != other._clientSecret)
                return false;
            return true;
        }
        public override bool Equals(object obj)
        {
            ClientBasicAuthentication asClientBasicAuthentication = obj as ClientBasicAuthentication;
            if (asClientBasicAuthentication == null)
                return false;
            return Equals(asClientBasicAuthentication);
        }
        public override int GetHashCode()
        {
            if (_hashCodeCalculated)
                return _hashCode;
            int hashCode = 0;
            unchecked
            {
                if (_encoding != null)
                    hashCode = (int)(hashCode * 397) ^ _encoding.GetHashCode();
                if (_clientId != null)
                    hashCode = (int)(hashCode * 397) ^ _clientId.GetHashCode();
                if (_clientSecret != null)
                    hashCode = (int)(hashCode * 397) ^ _clientSecret.GetHashCode();
                _hashCode = hashCode;
                _hashCodeCalculated = true;
                return hashCode;
            }
        }
        public static bool operator == (ClientBasicAuthentication left, ClientBasicAuthentication right)
        {
            return Equals(left, right);
        }
        public static bool operator != (ClientBasicAuthentication left, ClientBasicAuthentication right)
        {
            return !Equals(left, right);
        }
        public override string ToString()
        {
            return "ClientBasicAuthentication { ClientId = " + _clientId + ", ClientSecret = " + _clientSecret + " }";
        }
    }
}
```

Then we create the ClientBasic class and ClientBasicAuthentication class in the authorization header as a Base64 encoded string.

Write the AuthenticationFilter method of the ClientBasicAuthentication as shown below.

```
[AttributeUsage(AttributeTarget.Method, AllowMultiple = false)]
[ExtendedTrace]
[ExtendedTraceMethodCallsAspect]
public sealed class ClientBasicAuthenticationFilterAttribute : AuthenticationFilterAttribute
{
    private ILog _log = LogManager.GetLogger(typeof(ClientBasicAuthenticationFilterAttribute));
    private ClientBasicAuthenticationHandler _clientBasicAuthenticationHandler;
    public ClientBasicAuthenticationFilterAttribute(ClientBasicAuthenticationHandler clientBasicAuthenticationHandler)
        : base(clientBasicAuthenticationHandler.Encoding, clientBasicAuthenticationHandler.Realm)
        {
            _clientBasicAuthenticationHandler = clientBasicAuthenticationHandler;
            _log.Debug("ClientBasicAuthenticationFilterAttribute.ctor(ClientBasicAuthenticationHandler clientBasicAuthenticationHandler) - Initialized");
        }
    public override void AuthenticationChallenge(HttpAuthenticationChallengeContext context, ClientBasicAuthenticationCredentials credentials)
        {
            _log.Debug("ClientBasicAuthenticationFilterAttribute.AuthenticationChallenge(HttpAuthenticationChallengeContext context, ClientBasicAuthenticationCredentials credentials) - Entering");
            context.Result = new AuthenticationFailureResult("Unauthorized", context.Request);
            _log.Debug("ClientBasicAuthenticationFilterAttribute.AuthenticationChallenge(HttpAuthenticationChallengeContext context, ClientBasicAuthenticationCredentials credentials) - Exiting");
        }
    public override void AuthenticationFilterOnAuthorizationFailed(HttpAuthenticationContext context, ClientBasicAuthenticationCredentials credentials)
        {
            _log.Debug("ClientBasicAuthenticationFilterAttribute.AuthenticationFilterOnAuthorizationFailed(HttpAuthenticationContext context, ClientBasicAuthenticationCredentials credentials) - Entering");
            _log.Debug("ClientBasicAuthenticationFilterAttribute.AuthenticationFilterOnAuthorizationFailed(HttpAuthenticationContext context, ClientBasicAuthenticationCredentials credentials) - Exiting");
        }
    public override void AuthenticationFilterOnAuthorizationSucceeded(HttpAuthenticationContext context, ClientBasicAuthenticationCredentials credentials)
        {
            _log.Debug("ClientBasicAuthenticationFilterAttribute.AuthenticationFilterOnAuthorizationSucceeded(HttpAuthenticationContext context, ClientBasicAuthenticationCredentials credentials) - Entering");
            _log.Debug("ClientBasicAuthenticationFilterAttribute.AuthenticationFilterOnAuthorizationSucceeded(HttpAuthenticationContext context, ClientBasicAuthenticationCredentials credentials) - Exiting");
        }
    public override ClientBasicAuthenticationCredentials AuthenticationFilterParseCredentials(HttpAuthenticationContext context, ClientBasicAuthenticationCredentials credentials)
        {
            _log.Debug("ClientBasicAuthenticationFilterAttribute.AuthenticationFilterParseCredentials(HttpAuthenticationContext context, ClientBasicAuthenticationCredentials credentials) - Entering");
            var result = _clientBasicAuthenticationHandler.ParseCredentials(context, credentials);
            _log.Debug("ClientBasicAuthenticationFilterAttribute.AuthenticationFilterParseCredentials(HttpAuthenticationContext context, ClientBasicAuthenticationCredentials credentials) - Exiting");
            return result;
        }
    public override void AuthenticationFilterValidateCredentials(HttpAuthenticationContext context, ClientBasicAuthenticationCredentials credentials, ClientBasicAuthenticationCredentials validCredentials)
        {
            _log.Debug("ClientBasicAuthenticationFilterAttribute.AuthenticationFilterValidateCredentials(HttpAuthenticationContext context, ClientBasicAuthenticationCredentials credentials, ClientBasicAuthenticationCredentials validCredentials) - Entering");
            var result = _clientBasicAuthenticationHandler.ValidateCredentials(context, credentials, validCredentials);
            _log.Debug("ClientBasicAuthenticationFilterAttribute.AuthenticationFilterValidateCredentials(HttpAuthenticationContext context, ClientBasicAuthenticationCredentials credentials, ClientBasicAuthenticationCredentials validCredentials) - Exiting");
            if (result == null)
                return;
            if (result.Principal != null)
                context.Principal = result.Principal;
            if (result.ErrorResult != null)
                context.ErrorResult = result.ErrorResult;
            else
                _log.Error("ClientBasicAuthenticationFilterAttribute.AuthenticationFilterValidateCredentials(HttpAuthenticationContext context, ClientBasicAuthenticationCredentials credentials, ClientBasicAuthenticationCredentials validCredentials) - Unexpected result");
        }
    public override void OnAuthorization(HttpAuthenticationContext context, CancellationToken cancelToken)
        {
            _log.Debug("ClientBasicAuthenticationFilterAttribute.OnAuthorization(HttpAuthenticationContext context, CancellationToken cancelToken) - Entering");
            base.OnAuthorization(context, cancelToken);
            _log.Debug("ClientBasicAuthenticationFilterAttribute.OnAuthorization(HttpAuthenticationContext context, CancellationToken cancelToken) - Exiting");
        }
    public override void OnAuthorizationFailed(HttpAuthenticationContext context, ClientBasicAuthenticationCredentials credentials)
        {
            _log.Debug("ClientBasicAuthenticationFilterAttribute.OnAuthorizationFailed(HttpAuthenticationContext context, ClientBasicAuthenticationCredentials credentials) - Entering");
            _log.Debug("ClientBasicAuthenticationFilterAttribute.OnAuthorizationFailed(HttpAuthenticationContext context, ClientBasicAuthenticationCredentials credentials) - Exiting");
        }
    public override void OnAuthorizationSucceeded(HttpAuthenticationContext context, ClientBasicAuthenticationCredentials credentials)
        {
            _log.Debug("ClientBasicAuthenticationFilterAttribute.OnAuthorizationSucceeded(HttpAuthenticationContext context, ClientBasicAuthenticationCredentials credentials) - Entering");
            _log.Debug("ClientBasicAuthenticationFilterAttribute.OnAuthorizationSucceeded(HttpAuthenticationContext context, ClientBasicAuthenticationCredentials credentials) - Exiting");
        }
    public override Task OnAuthorizationAsyncAsync(HttpAuthenticationContext context, CancellationToken cancelToken)
        {
            _log.Debug("ClientBasicAuthenticationFilterAttribute.OnAuthorizationAsyncAsync(HttpAuthenticationContext context, CancellationToken cancelToken) - Entering");
            var result = base.OnAuthorizationAsyncAsync(context, cancelToken);
            _log.Debug("ClientBasicAuthenticationFilterAttribute.OnAuthorizationAsyncAsync(HttpAuthenticationContext context, CancellationToken cancelToken) - Exiting");
            return result;
        }
    public override bool AllowMultiple()
        {
            _log.Debug("ClientBasicAuthenticationFilterAttribute.AllowMultiple() - Entering");
            var result = base.AllowMultiple();
            _log.Debug("ClientBasicAuthenticationFilterAttribute.AllowMultiple() - Exiting");
            return result;
        }
    public override Task AuthenticationChallengeAsyncAsync(HttpAuthenticationChallengeContext context, ClientBasicAuthenticationCredentials credentials)
        {
            _log.Debug("ClientBasicAuthenticationFilterAttribute.AuthenticationChallengeAsyncAsync(HttpAuthenticationChallengeContext context, ClientBasicAuthenticationCredentials credentials) - Entering");
            var result = base.AuthenticationChallengeAsyncAsync(context, credentials);
            _log.Debug("ClientBasicAuthenticationFilterAttribute.AuthenticationChallengeAsyncAsync(HttpAuthenticationChallengeContext context, ClientBasicAuthenticationCredentials credentials) - Exiting");
            return result;
        }
    public override Task AuthenticationFilterOnAuthorizationFailedAsyncAsync(HttpAuthenticationContext context, ClientBasicAuthenticationCredentials credentials)
        {
            _log.Debug("ClientBasicAuthenticationFilterAttribute.AuthenticationFilterOnAuthorizationFailedAsyncAsync(HttpAuthenticationContext context, ClientBasicAuthenticationCredentials credentials) - Entering");
            var result = base.AuthenticationFilterOnAuthorizationFailedAsyncAsync(context, credentials);
            _log.Debug("ClientBasicAuthenticationFilterAttribute.AuthenticationFilterOnAuthorizationFailedAsyncAsync(HttpAuthenticationContext context, ClientBasicAuthenticationCredentials credentials) - Exiting");
            return result;
        }
    public override Task AuthenticationFilterOnAuthorizationSucceeded(HttpAuthenticationContext context, ClientBasicAuthenticationCredentials credentials)
        {
            _log.Debug("ClientBasicAuthenticationFilterAttribute.AuthenticationFilterOnAuthorizationSucceeded(HttpAuthenticationContext context, ClientBasicAuthenticationCredentials credentials) - Entering");
            _log.Debug("ClientBasicAuthenticationFilterAttribute.AuthenticationFilterOnAuthorizationSucceeded(HttpAuthenticationContext context, ClientBasicAuthenticationCredentials credentials) - Exiting");
            return null;
        }
    public override Task AuthenticationFilterParseCredentialsAsyncAsync(HttpAuthenticationContext context, ClientBasicAuthenticationCredentials credentials)
        {
            _log.Debug("ClientBasicAuthenticationFilterAttribute.AuthenticationFilterParseCredentialsAsyncAsync(HttpAuthenticationContext context, ClientBasicAuthenticationCredentials credentials) - Entering");
            var result = base.AuthenticationFilterParseCredentialsAsyncAsync(context, credentials);
            _log.Debug("ClientBasicAuthenticationFilterAttribute.AuthenticationFilterParseCredentialsAsyncAsync(HttpAuthenticationContext context, ClientBasicAuthenticationCredentials credentials) - Exiting");
            return result;
        }
    public override Task AuthenticationFilterValidateCredentialsAsyncAsync(HttpAuthenticationContext context, ClientBasicAuthenticationCredentials credentials, ClientBasicAuthenticationCredentials validCredentials)
        {
            _log.Debug("ClientBasicAuthenticationFilterAttribute.AuthenticationFilterValidateCredentialsAsyncAsync(HttpAuthenticationContext context, ClientBasicAuthenticationCredentials credentials, ClientBasicAuthenticationCredentials validCredentials) - Entering");
            var result = base.AuthenticationFilterValidateCredentialsAsyncAsync(context, credentials, validCredentials);
            _log.Debug("ClientBasicAuthenticationFilterAttribute.AuthenticationFilterValidateCredentialsAsyncAsync(HttpAuthenticationContext context, ClientBasicAuthenticationCredentials credentials, ClientBasicAuthenticationCredentials validCredentials) - Exiting");
            return result;
        }
    public override Task OnAuthorizationAsyncAsync(HttpAuthenticationContext context, CancellationToken cancelToken)
        {
            _log.Debug("ClientBasicAuthenticationFilterAttribute.OnAuthorizationAsyncAsync(HttpAuthenticationContext context, CancellationToken cancelToken) - Entering");
            var result = base.OnAuthorizationAsyncAsync(context, cancelToken);
            _log.Debug("ClientBasicAuthenticationFilterAttribute.OnAuthorizationAsyncAsync(HttpAuthenticationContext context, CancellationToken cancelToken) - Exiting");
            return result;
        }
    public override void OnAuthorizationFailedAsyncAsync(HttpAuthenticationContext context, ClientBasicAuthenticationCredentials credentials)
        {
            _log.Debug("ClientBasicAuthenticationFilterAttribute.OnAuthorizationFailedAsyncAsync(HttpAuthenticationContext context, ClientBasicAuthenticationCredentials credentials) - Entering");
            _log.Debug("ClientBasicAuthenticationFilterAttribute.OnAuthorizationFailedAsyncAsync(HttpAuthenticationContext context, ClientBasicAuthenticationCredentials credentials) - Exiting");
        }
    public override void OnAuthorizationSucceeded(HttpAuthenticationContext context, ClientBasicAuthenticationCredentials credentials)
        {
            _log.Debug("ClientBasicAuthenticationFilterAttribute.OnAuthorizationSucceeded(HttpAuthenticationContext context, ClientBasicAuthenticationCredentials credentials) - Entering");
            _log.Debug("ClientBasicAuthenticationFilterAttribute.OnAuthorizationSucceeded(HttpAuthenticationContext context, ClientBasicAuthenticationCredentials credentials) - Exiting");
        }
}
```

Write the Load Method in the ClientBasic class and ClientBasicAuthentication class in the authorization header as a Base64 encoded string.

Write the ParseCredentials method in the ClientBasicAuthentication class.

```
[AttributeUsage(AttributeTarget.Method, AllowMultiple = false)]
[ExtendedTrace]
[ExtendedTraceMethodCallsAspect]
public sealed class ClientBasicAuthenticationHandler : IEquatable<ClientBasicAuthenticationHandler>
{
    private ILog _log = LogManager.GetLogger(typeof(ClientBasicAuthenticationHandler));
    private Encoding _encoding;
    private ClientBasicAuthenticationCredentials _credentials;
    private ClientBasicAuthenticationCredentials _validCredentials;
    private ClientBasicAuthenticationCredentials _defaultCredentials;
    private ClientBasicAuthenticationCredentials[] _validCredentialsArray;
    private bool _allowMultiple;
    private int _hashCode;
    private bool _hashCodeCalculated;
    public ClientBasicAuthenticationHandler(Encoding encoding, ClientBasicAuthenticationCredentials[] validCredentialsArray, ClientBasicAuthenticationCredentials defaultCredentials, bool allowMultiple)
        {
            _log.Debug("ClientBasicAuthenticationHandler.ctor(Encoding encoding, ClientBasicAuthenticationCredentials[] validCredentialsArray, ClientBasicAuthenticationCredentials defaultCredentials, bool allowMultiple) - Entering");
            if (encoding == null)
                throw new ArgumentNullException("encoding");
            if (validCredentialsArray == null)
                throw new ArgumentNullException("validCredentialsArray");
            if (defaultCredentials == null)
                throw new ArgumentNullException("defaultCredentials");
            _encoding = encoding;
            _validCredentialsArray = validCredentialsArray;
            _defaultCredentials = defaultCredentials;
            _allowMultiple = allowMultiple;
            _log.Debug("ClientBasicAuthenticationHandler.ctor(Encoding encoding, ClientBasicAuthenticationCredentials[] validCredentialsArray, ClientBasicAuthenticationCredentials defaultCredentials, bool allowMultiple) - Exiting");
        }
    public ClientBasicAuthenticationHandler(ClientBasicAuthenticationHandler other)
        : this(other._encoding, other._validCredentialsArray, other._defaultCredentials, other._allowMultiple)
        {
            if (other == null)
                throw new ArgumentNullException("other");
            _credentials = other._credentials;
            _hashCode = other._hashCode;
            _hashCodeCalculated = other._hashCodeCalculated;
        }
    private ClientBasicAuthenticationCredentials[] _validCredentialsArray;
    private ClientBasicAuthenticationCredentials _defaultCredentials;
    private bool _allowMultiple;
    private ILog _log;
    private Encoding _encoding;
    private ClientBasicAuthenticationCredentials _credentials;
    private int _hashCode;
    private bool _hashCodeCalculated;
    public Encoding Encoding
        {
            get
            {
                return _encoding;
            }
            set
            {
                if (value == null)
                    throw new ArgumentNullException("value");
                _encoding = value;
                _credentials = null;
                _hashCode = 0;
                _hashCodeCalculated = false;
            }
        }
    public ClientBasicAuthenticationCredentials[] ValidCredentialsArray
        {
            get
            {
                return _validCredentialsArray;
            }
            set
            {
                if (value == null)
                    throw new ArgumentNullException("value");
                _validCredentialsArray = value;
                _credentials = null;
                _hashCode = 0;
                _hashCodeCalculated = false;
            }
        }
    public ClientBasicAuthenticationCredentials DefaultCredentials
        {
            get
            {
                return _defaultCredentials;
            }
            set
            {
                if (value == null)
                    throw new ArgumentNullException("value");
                _defaultCredentials = value;
                _credentials = null;
                _hashCode = 0;
                _hashCodeCalculated = false;
            }
        }
    public bool AllowMultiple
        {
            get
            {
                return _allowMultiple;
            }
            set
            {
                _allowMultiple = value;
                _credentials = null;
                _hashCode = 0;
                _hashCodeCalculated = false;
            }
        }
    public ClientBasicAuthenticationCredentials Credentials
        {
            get
            {
                if (_credentials == null)
                    _credentials = _validCredentialsArray.Single(c => c.Equals(_defaultCredentials));
                return _credentials;
            }
            private set
            {
                _credentials = value;
                _hashCode = 0;
                _hashCodeCalculated = false;
                _log.Debug("ClientBasicAuthenticationHandler.Credentials.set(ClientBasicAuthenticationCredentials value) - Exiting");
            }
        }
    public bool Equals(ClientBasicAuthenticationHandler other)
        {
            if (other == null)
                return false;
            if (_encoding != other._encoding)
                return false;
            if (_validCredentialsArray != other._validCredentialsArray)
                return false;
            if (_defaultCredentials != other._defaultCredentials)
                return false;
            if (_allowMultiple != other._allowMultiple)
                return false;
            if (_credentials != other._credentials)
                return false;
            return true;
        }
    public override bool Equals(object obj)
        {
            ClientBasicAuthenticationHandler asClientBasicAuthenticationHandler = obj as ClientBasicAuthenticationHandler;
            if (asClientBasicAuthenticationHandler == null)
                return false;
            return Equals(asClientBasicAuthenticationHandler);
        }
    public override int GetHashCode()
        {
            if (_hashCodeCalculated)
                return _hashCode;
            int hashCode = 0;
            unchecked
            {
                if (_encoding != null)
                    hashCode = (int)(hashCode * 397) ^ _encoding.GetHashCode();
                if (_validCredentialsArray != null)
                    hashCode = (int)(hashCode * 397) ^ _validCredentialsArray.GetHashCode();
                if (_defaultCredentials != null)
                    hashCode = (int)(hashCode * 397) ^ _defaultCredentials.GetHashCode();
                if (_credentials != null)
                    hashCode = (int)(hashCode * 397) ^ _credentials.GetHashCode();
                _hashCode = hashCode;
                _hashCodeCalculated = true;
                return hashCode;
            }
        }
    public static bool operator == (ClientBasicAuthenticationHandler left, ClientBasicAuthenticationHandler right)
        {
            return Equals(left, right);
        }
    public static bool operator != (ClientBasicAuthenticationHandler left, ClientBasicAuthenticationHandler right)
        {
            return !Equals(left, right);
        }
    public override string ToString()
        {
            return "ClientBasicAuthenticationHandler { Encoding = " + _encoding + ", ValidCredentialsArray = " + _validCredentialsArray + ", DefaultCredentials = " + _defaultCredentials + ", AllowMultiple = " + _allowMultiple + ", Credentials = " + _credentials + " }";
        }
    public ClientBasicAuthenticationCredentials[] ParseCredentials(HttpAuthenticationContext context, ClientBasicAuthenticationCredentials credentials)
        {
            _log.Debug("ClientBasicAuthenticationHandler.ParseCredentials(HttpAuthenticationContext context, ClientBasicAuthenticationCredentials credentials) - Entering");
            var result = new ClientBasicAuthenticationCredentials[] { credentials };
            _log.Debug("ClientBasicAuthenticationHandler.ParseCredentials(HttpAuthenticationContext context, ClientBasicAuthenticationCredentials credentials) - Exiting");
            return result;
        }
    public ClientBasicAuthenticationCredentials[] ValidateCredentials(HttpAuthenticationContext context, ClientBasicAuthenticationCredentials credentials, ClientBasicAuthenticationCredentials validCredentials)
        {
            _log.Debug("ClientBasicAuthenticationHandler.ValidateCredentials(HttpAuthenticationContext context, ClientBasicAuthenticationCredentials credentials, ClientBasicAuthenticationCredentials validCredentials) - Entering");
            var result = new ClientBasicAuthenticationCredentials[] { validCredentials };
            _log.Debug("ClientBasicAuthenticationHandler.ValidateCredentials(HttpAuthenticationContext context, ClientBasicAuthenticationCredentials credentials, ClientBasicAuthenticationCredentials validCredentials) - Exiting");
            return result;
        }
}
```

Step 2. Implement Implements the same Client class and use that class for this same.

Leave a Reply

Your email address will not be published. Required fields are marked *

Comment *

Name * Email * Website

Post Comment

About Us | Privacy Policy | Contact | DOTNET Tricks | Angular Tutorial | ASP.NET Core Interview Questions | ASP.NET MVC Tutorial | ASP.NET Web API Tutorial | ASP.NET Core Tutorial | ASP.NET Core Interview Questions | ASP.NET Core Tutorial | ASP.NET Core Interview Questions | ASP.NET Core Tutorial | ASP.NET Core Interview Questions